CStar Gateway: Augmenting Public Cloud Infrastructure for Heterogeneous Network Function Virtualization

Haonan Li[‡], Tian Pan[‡□], Jin Ke[‡], Baohai Hu[‡], Changgang Zheng[‡], Enge Song[‡], Zhi Xu[‡], Ye Yang[‡], Bowen Yang[‡], Donglin Lai[‡], Yisong Qiao[‡], Bengbeng Xue[‡], Jianyuan Lu[‡], Xiaoqing Sun[‡], Shize Zhang[‡] Zihao Fan[‡], Mingxin Li[‡], Yang Song[‡□], Jun Liang[‡], Xionglie Wei[‡], Biao Lyu[§‡], Rong Wen[¶‡], Zhigang Zong[‡], Jiao Zhang[†], Tao Huang[†], Shunmin Zhu[‡□]

[‡]*Alibaba Cloud* [§]*Zhejiang University* [¶]*Fudan University* [†]*Purple Mountain Laboratories*

Abstract

Major cloud providers often build NFV products by reusing existing architectures. These designs were originally developed and optimized for tenant virtual machines (VMs) that operate at network endpoints. In contrast, network function (NF) VMs function as intermediate forwarding nodes shared by multiple tenants, which have very different resource demands. Directly applying tenant-oriented designs creates mismatches between infrastructure capabilities and NF requirements. A common case arises when NF VMs serve many tenants: they typically exhaust vNIC and sessions on vSwitches well before CPU resources are saturated. At this point, NF VMs cannot accept additional traffic despite having idle CPU cycles. Providers usually address this by scaling out or scaling up NF VMs, which wastes resources and increases cost. This is just one example, where numerous other mismatches remain. To address these, we present CStar Gateway. CStar Gateway shifts NF VM multi-tenancy support from vSwitches into NF VMs, which reduces vNIC and session bottlenecks with minimal changes to the existing cloud infrastructure. CStar Gateway also identifies and redirects I/O- or CPU-intensive flows to FPGA-based NFs, increasing the service capacity. In addition, CStar Gateway takes over NF VM elasticity support from vSwitch, simplifying and accelerating the scaling process, and thereby enhancing overall system flexibility. Deployment results show that the design improves CPU and I/O utilization of NF VMs by at least 5x and reduces NF cluster capital expenditure by 80.43% to 91.34%.

1 Introduction

Network Function Virtualization (NFV) in public clouds decouples Network Functions (NFs) from proprietary hardware to enhance flexibility and resilience [25]. However, NFV instances face challenges in managing state while sustaining line-rate performance during scale-up [9, 19, 20, 30] and scale-out [13, 21, 22, 27]. In addition, NFV systems must accommodate high tenant traffic variability and distinct SLAs with elasticity while ensuring strict isolation [15, 16, 26].

To address challenges, cloud service providers have reported numerous NFV implementations [3, 5, 7, 17, 28]. They maximize reuse of existing infrastructure to avoid redundant development. For example, Azure integrates basic NF capabilities like ACLs in its virtual switch (vSwitch) [7], and builds a DPU-accelerated shared resource pool to handle overflow traffic exceeding the vSwitch's capacity [3]. In contrast, AWS Hyperplane [17] and Alibaba CyberStar (*a.k.a.*, CStar) [28] use standard VMs to implement NFs, enabling more complex capabilities (*e.g.*, L7-Load Balancer (LB), VPN). These NF VMs fully leverage existing cloud infrastructure for elasticity and fault tolerance. Moreover, NF VMs are decoupled from the vSwitch, ensuring that sudden traffic surges or failures in NF VMs do not compromise core VPC routing.

CStar is a multi-tenant, multi-service, cloud-based NFV architecture using existing tenant VMs [2] to accommodate NFs. NF VMs coexist with tenant VMs on the same vSwitch and can migrate freely. CStar supports both scaling out multiple VMs to serve one tenant and sharing a single VM among multiple tenants for resource efficiency. To preserve isolation across tenants' demands in a shared environment, CStar provisions multiple vNICs on a single NF VM, each carrying NF's demand-specific traffic throttling configurations, and routes traffic from different tenants' demands through separate vNICs to the NF processing logic.

Despite the advantages of CStar, we still encounter several technical issues. The first is that some NF VMs exhaust the vNICs or sessions provided by vSwitch, even though their CPU or I/O utilization remains low. This prevents them from accepting additional traffic. As a result, we are forced to provision more NF VMs to meet growing traffic demands, which unreasonably increases our costs. Our analysis shows that the root cause lies in the mismatch between the heterogeneous resource requirements of NF VMs (*e.g.*, vNICs, sessions) and the resource capabilities offered by vSwitches to VMs. Since tenant VMs constitute the majority of cloud workloads, cloud infrastructure is optimized for them. However, NF VMs differ significantly from tenant VMs in two aspects: (1) tenant VMs typically require only 1-2 vNICs, while NF VMs often

[□]Co-corresponding authors.

need hundreds of vNICs as network intermediate nodes; (2) tenant VMs consume session resources only for their own traffic, whereas NF VMs, positioned at traffic aggregation points, must maintain sessions for a large number of concurrent flows.

The second issue lies in the imbalance between I/O and CPU resource utilization within individual NF VMs. Some NF VMs exhibit high I/O utilization but low CPU usage, while others frequently suffer from CPU overload due to heavy-hitter traffic. This disparity arises from the different packet processing models of different NFs: some are I/O intensive (*e.g.*, Transit Router (TR) [1]), whereas others are more CPU-intensive (*e.g.*, VPN). As a result, NF VMs consume resources in varying proportions.

The third issue is that the elasticity support of NF VMs differs significantly from that of tenant VMs. Each tenant VM has a unique IP, so scaling out tenant VMs only requires assigning new IPs via the controller for direct access to the newly created VMs. In contrast, NF VM scaling is hidden behind a service IP assigned to the tenant, with traffic distributed to NF VMs via vSwitch hashing. Since NF VMs often maintain state, consistent hashing [6,14] on the vSwitch is preferable to reduce state inconsistency during scaling. This adds significant performance overhead to the vSwitch.

To address these issues without causing changes to the existing cloud infrastructure, we propose a "minimally invasive" solution by introducing a programmable gateway named CStar Gateway that sits between tenant VMs and NF VMs. To eliminate vNIC exhaustion, the gateway encapsulates an NF-specific configuration identifier (*i.e.*, fake vNIC ID) into an extra GENEVE tunnel [11] and delivers it to the NF VM. The NF VM extracts the identifier and executes the corresponding traffic throttling operations (*e.g.*, pass, meter, or drop) that were previously handled by the vSwitch. As a result, NF VMs no longer need multiple vNICs from the vSwitch to differentiate NF-specific demands for different tenants. To address session exhaustion, the gateway relays traffic from tenant VMs and aggregates multiple flows with distinct 5-tuples into a single flow by encapsulating them into the GENEVE tunnel. This enables the vSwitch to track only one session per NF VM. To address the I/O-CPU imbalance in NF VMs under bandwidth-heavy and elephant flows, we offload these workloads to FPGAs, whose resource profile better matches their demands. To reduce the burden on the vSwitch in supporting the elasticity of NF VMs, we also offload the consistent hashing functionality from the vSwitch to the gateway. So far, CStar Gateway has been used for about two years. We share the experiences and lessons learned from its development and deployment.

Our major contributions are summarized as follows.

- We present CStar Gateway, a minimally invasive architecture that shifts multi-tenant, multi-demand support from the vSwitch to NF VMs using tunnel encapsulation, without modifying the vSwitch. This removes vNIC and session limits, enabling NF VMs to fully utilize idle CPU and I/O resources, improving per-NF VM resource utilization by up to 5×.
- We design a hybrid data plane that offloads CPU- and I/O-intensive flows to FPGA for line-rate performance, while keeping other flows on elastic NF VMs. This approach combines the FPGA's high throughput with VM's elasticity.
- We offload NF VM elasticity support from vSwitch to CStar Gateway, simplifying and accelerating the scaling process, enhancing overall system flexibility.
- After deployment across three availability zones (AZs), NF infrastructure costs are reduced by 80-91% by efficiently serving increased traffic with fewer NF VMs. With CStar Gateway, Alibaba Cloud is no longer plagued by the long-standing challenge of NFV profitability.

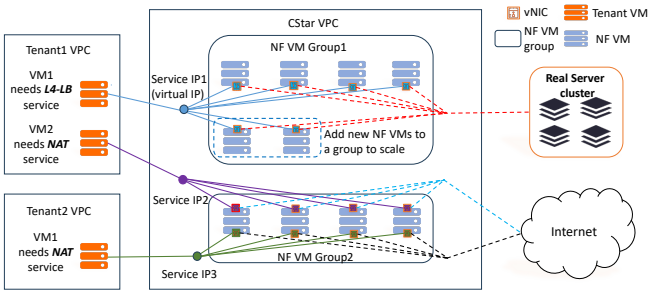

Figure 1: Cloud-based NFV deployment with CStar.

2 Cloud-based NFV Deployment

Multi-tenancy and Multi-demand NFs. As illustrated in Fig. 1, Alibaba Cloud's NF platform named Cstar serves multiple tenants with heterogeneous NF demands, comprising two dimensions: (1) NF type (*e.g.*, LB, NAT); (2) NF access control (traffic admitted only from designated source IPs, subject to rate limits). For example, as shown in Fig.1, Tenant 1 requires LB accessed only with VM1 and NAT accessed only with VM2; Tenant 2 has one NAT demand. As a network middlebox, the NF VM concurrently processes multi-tenant, multi-demand traffic. To ensure isolation, we provide stateful NF services at (tenant id, service id) granularity. Tenant id identifies the tenant, service id distinguishes between different NF demands. We combine vSwitch infrastructure support (*e.g.*, ACLs, metering) and in-VM programmable logic (*e.g.*, LB policy) to fulfill NF demands. For Tenant 1's LB demand: 1) vSwitch allows only VM1's IPs; 2) NF VM distributes traffic via least-connections. Given the scale of traffic, multiple NF VMs are grouped to share the load. Tenants access services via a virtual Service IP. Traffic is distributed across the group via ECMP. This group architecture provides rapid elasticity: NF VMs can be dynamically added or removed to match traffic without service interruption.

Tenant NF Access Control. To rapidly roll out NF services and satisfy tenants' NF access control requirements, we leverage the existing capabilities of vNICs provided to NF VMs

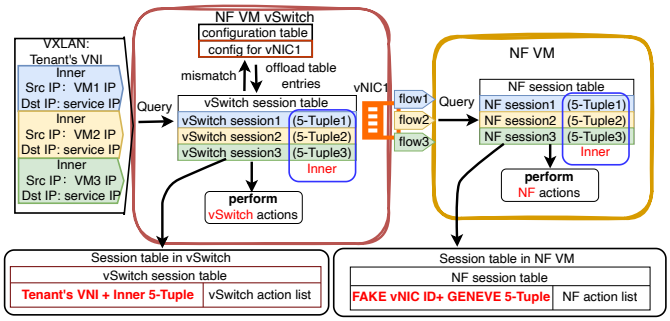

Figure 2: Role of session in vSwitch.

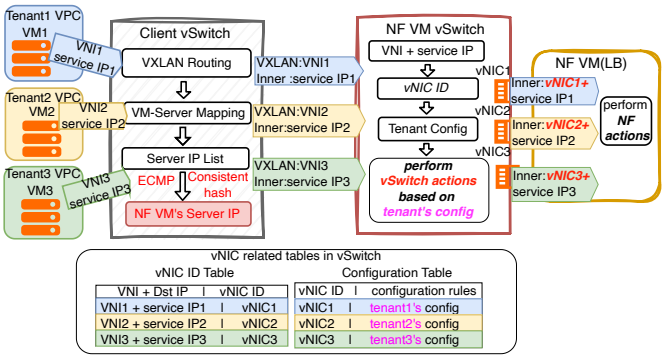

Figure 3: Packet journey from the tenant VM to the NF VM.

by the vSwitch. Associating configuration information, such as traffic access rules and rate limits, with vNICs is a common practice among cloud providers, including Azure, to enable VM communication while enforcing tenant requirements. This approach aligns well with our need for per-tenant, per-NF access control. In Alibaba Cloud's architecture, specifically, each VM (including NF VMs) is assigned one or more vNICs. Each vNIC holds specific configurations, such as security group rules (*e.g.*, allow 10.1.0.0/16) and rate limits (*e.g.*, 1 Gbps). The vSwitch applies these configurations to process incoming traffic (pass/drop/meter) before forwarding it to the VM. To serve Tenant 1's LB demand, for example: 1) A dedicated vNIC is created on the NF VM's vSwitch; 2) It is configured with Tenant 1's ACL and meter demands; 3) Routing steers all Tenant 1 LB traffic through this vNIC. Importantly, the vSwitch handles only access control (ACL, metering). The core NF logic (*e.g.*, LB, NAT) is implemented inside the NF VM via programmable service code.

High-performance NF Traffic Processing. To ensure line-rate processing, every flow is assigned one session entry, created on its first packet, and reused for all subsequent packets. As shown in Fig. 2, the vSwitch looks up the target vNIC and its configs (ACL, metering) and creates a session entry storing: 1) vNIC ID (where to steer); 2) precomputed actions (pass/drop/meter) on the first packet. Later packets match only the session and perform cached actions to skip configuration lookup and compute actions. This design embodies the fast/slow-path separation principle. NF VMs also use session design. But the configs and session entries in NF VM are different from those in vSwitches. In CStar architecture, no NF packet is forwarded without a session, ensuring all traffic runs at line rate.

Routing of Tenant NF Traffic to NF VMs. To illustrate how Alibaba Cloud uses multi-vNIC in vSwitch to deliver multi-tenant and multi-demand NFs, we trace the end-to-end path from tenant traffic generation to NF VM processing (Fig. 3). When three tenants (Tenant 1–3) each have a distinct NF, the cloud provider allocates a dedicated Service IP to each. NF VMs share the same underlying infrastructure as tenant VMs, and their traffic is routed in the same way. In Client-side vSwitch, when receiving a packet, it first checks the VXLAN routing table. If the destination matches a Service IP, the packet is mapped to an internal cstar VPC (invisible to tenants), and the tenant's VNI is preserved. The vSwitch then looks up the target NF VM group using (VNI, Service IP) and returns the list of backend server IPs hosting NF VMs in that group. Finally, it applies ECMP on the packet's inner 5-tuple to select a backend server and forwards the packet to the corresponding NF VM host.

When receiving a packet, the NF-side vSwitch extracts (Tenant VNI, Service IP) from the packet, looks up the vNIC ID in the vNIC ID table, and retrieves the corresponding vNIC (*e.g.*, vNIC1 for Tenant 1's demand). It then fetches the tenant-specific configuration (security group, rate limit) from the configuration table based on the vNIC ID (*e.g.*, tenant1's config), and conducts per-tenant NF access control (pass/drop/meter/decap) accordingly. The packet is then delivered through the selected vNIC to the NF VM, which executes the NF service logic (*e.g.*, SNAT/DNAT, LB).

Elastic NF Management. In CStar, a controller creates and manages NF VM resources. Upon receiving a tenant demand for creating an NF service, it assigns a group of NF VMs to the tenant. To ensure high availability, the controller implements shuffle sharding [18] to distribute NF VMs across different servers, reducing the risk of "query of death" [23]. After resource allocation, the controller configures routing tables in the vSwitches, managing tenant VM traffic and NF VM traffic to ensure correct traffic delivery. When a tenant NF traffic increases, the controller can horizontally scale NF performance by adding more NF VMs to the group (Fig.1). Specifically, in CStar, "everything is a VM and a VM can go everywhere". A tenant vNIC can coexist with another tenant vNIC on the same NF VM, and NF VMs can coexist with tenant VMs on the same server. The elasticity of the cloud enables CStar to deliver high availability and resource efficiency.

3 NFV-Infrastructure Mismatches

Our cloud-based NFV reuses existing public cloud infrastructure to inherit cloud benefits and avoid redundant development. To launch the service quickly, we directly leveraged the native vNIC's traffic access control capabilities provided by the vSwitch and used VM types with larger vNIC counts to deploy NFs. While this worked for a small number of tenants, rapid growth made scaling by simply adding VMs too costly. We discovered significant resource imbalances where vNICs

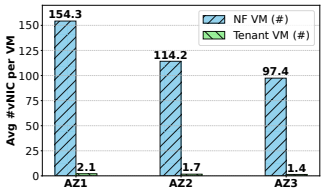

Figure 4: Average vNICs on NF VMs and tenant VMs.

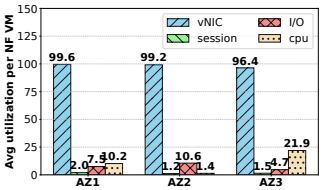

Figure 5: Resource utilization in VPC NAT NF VM.

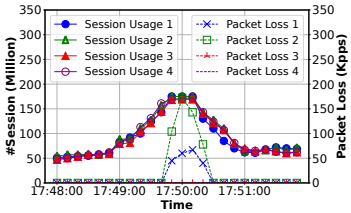

Figure 6: Packet loss due to vSwitch session exhaustion.

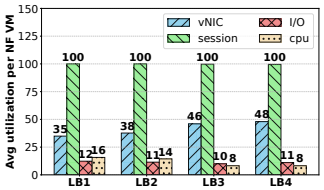

Figure 7: Resource utilization in LB NF VM.

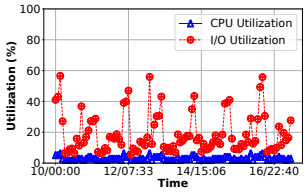

Figure 8: I/O saturation despite low CPU utilization.

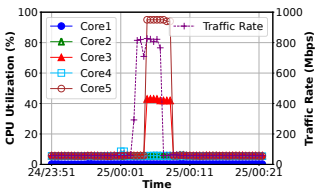

Figure 9: CPU core overload due to heavy-hitter flows.

were exhausted while CPU and bandwidth remained available. This is because current cloud infrastructure, optimized for tenant VMs at network endpoints, fundamentally mismatches the resource consumption patterns and architectural demands of NF VMs acting as intermediate traffic forwarders.

To accommodate the broad range of tenant demands, Alibaba Cloud has introduced multiple specialized NF VM types (*e.g.*, Layer-4 LB, NAT, TR). In production environments, we employ a dedicated platform to monitor and record various resource utilization metrics for these VMs that implement NFs. These NF VM variants exhibit distinct resource consumption, directly correlated with the characteristics of the traffic they process. Based on the data recorded by this platform, we analyze several representative NF VM types processing real-world service workloads in the following sections. These case studies empirically reveal and quantify the resource mismatch between NF VMs and general-purpose tenant VMs.

Mismatch 1: NF VM's massive occupation of vNICs provided by vSwitch. Our production data(Fig. 4) shows that in 3 AZs, NF VMs consume significantly more vNICs than tenant VMs, averaging over 97 per NF VM compared to 2 per tenant VM.

Workload profiling shows that the NF VMs which exhaust the vNICs are primarily used to implement the VPC NAT. The VPC NAT resolves IP address conflicts when multiple tenant VPCs communicate with a shared service VPC. A common example is Alibaba Cloud's Simple Log Service (SLS). SLS runs in a dedicated, centrally managed VPC whose internal IP ranges frequently overlap with the private CIDR blocks used by tenant VPCs. VPC NAT NF VMs must bind one vNIC per tenant demand to maintain isolation, rapidly reaching the vSwitch vNIC quota limit. Despite this, their negligible workload—consisting of small, periodic log uploads—leaves CPU, memory, and I/O grossly underutilized. We profiled VPC NAT NF VMs across 3 AZs (different from AZs in Fig. 4). As shown in Fig. 5, revealing that all sampled VMs exhausted vNICs while remaining largely idle on other resources. Once vNICs are exhausted, NF VMs can no longer serve additional tenant demands even if other resources remain idle, forcing operators to deploy additional costly and underutilized NF VMs as a workaround.

The vNIC quota per NF VM is constrained (*e.g.*, maximum 400). This limit is not due to hardware incapacity, but to economic design. Specifically, we leverage virtualization technology (*e.g.*, virtio) on SmartNICs to provide multiple vNICs for VMs. However, implementing each vNIC consumes SmartNIC resources, such as storage and bandwidth. As SmartNICs support various tasks, the storage resources available for implementing vNICs are severely limited. This results in the vSwitch on each SmartNIC being able to support only about 4K vNICs per server. Because NF VMs share this vNIC pool with tenant VMs, allocating more to NF VMs would directly reduce tenant VM density and, in turn, per-server revenue. Quotas are therefore needed to ensure system-wide fairness and business efficiency. We experimented with network-optimized VMs, which provide more vNICs and higher bandwidth but with fewer CPU and memory resources to minimize waste. More performance data of different VM types we have used could be get in Appendix .B. While these efforts help, NF VMs still hit the quota, albeit slightly later. We also chose not to design a custom VM for VPC NAT, given 1) high development cost, 2) limited market demand (normal tenants won't buy it), and 3) poor cost-effectiveness (NF VMs account for «1% of all VMs).

At provisioning, the vSwitch assigns fixed per-VM quotas for vNICs, sessions, and I/O. This static allocation guarantees tenant isolation, predictable performance, and transparent billing. To avoid potential resource underutilization, tenants should choose VM flavors (e.g., network-optimized instances) that match their needs. The platform avoids dynamic or shared pools. While they improve average utilization, their risks including resource hijacking, cross-tenant interference, and unpredictable performance outweigh the benefits in commercial multi-tenant environments.

Mismatch 2: NF VMs require far more sessions than tenant VMs. In production, session table exhaustion on NF VMs frequently causes packet loss and service degradation. For example, in Fig. 6's 4-VM group, two instances hit 100% capacity with severe drops, while the other two are nearing capacity and risk failure. ession exhaustion primarily affects

LB NF VMs serving (1) microservices accessing backend storage and (2) IoT fleets connecting to VPC services (e.g., storage, compute) [29]. Both workloads generate bursts of short-lived flows (often exceeding 1M concurrent sessions within seconds) that rapidly fill vSwitch session tables. Session exhaustion occurs even while other resources remain underutilized: I/O is low (small packets), CPU is mostly idle (no L7 parsing), and vNIC usage is moderate (limited tenants per LB). Consequently, the vSwitch session quota restricts LB throughput. Fig. 7 confirms this, showing four LB VMs hitting session limits while compute and bandwidth remain low, proving session capacity is the dominant bottleneck.

Once vSwitch session tables exhaust, new flows drop despite abundant CPU, memory, and I/O. The current workaround by scaling out NF VMs incurs additional resource costs. Meanwhile, raising session quotas or creating "high-session" VM types is infeasible due to the same economic and architectural constraints limiting vNIC scaling.

Mismatch 3: NF's I/O bandwidth can saturate despite low CPU utilization. Profiling (Fig. 8) shows some NF VMs nearing peak I/O utilization with only 6% CPU usage, demonstrating that I/O can become the primary bottleneck. TR NF VM exemplifies this issue. It handles bandwidth-heavy inter-VPC traffic (e.g., HD video, cloud sync) using stateless, L3-only forwarding. Since it involves no deep packet inspection, encryption, or L7 processing, CPU requirements are minimal. However, because cloud VMs (even I/O-optimized ones) are inherently compute-centric, scaling TR VMs to meet I/O demands leaves CPU idle, highlighting a fundamental mismatch between bandwidth-centric workloads and compute-centric VM architectures.

Although both interconnect VPCs, TR forwards all traffic (generating high bandwidth), whereas VPC NAT only processes IP conflicts. To maintain quality, TR limits tenant density. Furthermore, unlike load balancers (LB) that suffer session explosions, TR handles long-lived, high-throughput flows with minimal session use. Consequently, facing moderate vNIC and negligible session pressures, I/O bandwidth emerges as TR's primary scaling constraint. Since NF VM pricing bundles CPU, memory, and I/O, scaling purely for I/O forces operators to pay for idle CPUs. At Tbps-scale deployments, this inflated cost compresses profit margins, making current VM-based scaling economically unsustainable.

Mismatch 4: NF CPU can be overloaded by heavy-hitter flows. Although RSS distributes NF traffic across CPU cores, persistent elephant flows can be pinned to a single core within an NF VM (Fig. 9), causing saturation and collateral packet loss for all flows on that core. This unpredictable risk universally affects all NF types (e.g., LB, VPC NAT, TR). Furthermore, because vNIC metering enforces bandwidth caps at the aggregate level rather than per-flow, an elephant flow can saturate a CPU core while remaining under the vNIC's overall limit.

Mismatch 5: NF VM elasticity is more complex than for tenant VMs. As noted Sec. 2, the tenent-side vSwitch distributes NF traffic across an NF VM group using ECMP hashing. When the group scales (VMs added/removed), the hash value changes, redirecting existing flows to different NF VMs. Since session state resides only on the original NF VM, the state must be migrated for flows to continue processing correctly. To mitigate disruption, consistent hashing is used on vSwitch to minimize flow remapping during scaling events. However, consistent hashing consumes significantly more vSwitch CPU and memory than standard hashing. In contrast, tenant VMs generally maintain stable 1:1 IP-to-VM mappings, requiring no hashing at all. Scaling an NF VM group triggers a control-plane storm. The controller must push updated forwarding rules to many distributed tenant-side vSwitches. This process is also very slow, taking 10s. As a result, NF scaling remains reactive, not predictive, and fails to meet the sub-second agility expected in cloud-native environments

4 NFV Overhead Outsourcing

4.1 Remove vNIC/Session Limits by Gateway

Aggregating Multi-tenant and Multi-demand vNICs. To overcome vNIC limits, CStar Gateway aggregates tenant-specific vNICs into a single vNIC per NF VM. It offloads tenant configurations (e.g., security groups, rate limits) from the vSwitch directly to the NF VM, leaving the vSwitch with one lightweight, tenant-agnostic vNIC.

The NF VM utilizes a logical identifier called a "fake vNIC ID" to distinguish between tenant configurations, replacing the physical vNIC ID in NF VM-side vSwitch. It is called "fake" because it serves only as a key to differentiate and index tenant demand-specific configurations inside the NF VM, with no corresponding physical vNIC. To maintain compatibility, the CStar Gateway offloads the lookup mechanism, mapping the (VNI, Service IP) tuple directly to the fake vNIC ID. To deliver this ID, the gateway encapsulates the original packet with a GENEVE header containing the ID, further wrapped in a VXLAN header for transport.

With tenant demand-specific configurations migrated, the NF VM-side vSwitch can no longer perform per-tenant demand access control, which is now fully offloaded to NF VMs. As shown in Fig. 10, upon receiving a packet, the NF VM extracts the fake vNIC ID from the GENEVE, retrieves corresponding configurations, and executes vSwitch-equivalent NF access control actions before proceeding to the NF logic. This design preserves functionality while effectively eliminating vNIC limits at the NFVM-side vSwitch level.

As shown in Fig. 10, the new architecture requires only one vNIC per NF VM to serve all tenant demands, compared to the previous three in Fig. 3. This design achieves fine-grained multi-tenant and multi-demand NF access control without exhausting vSwitch vNICs.

Offloads Session Management. As shown in Fig. 11, CStar Gateway encapsulates payloads with an inner GEN-

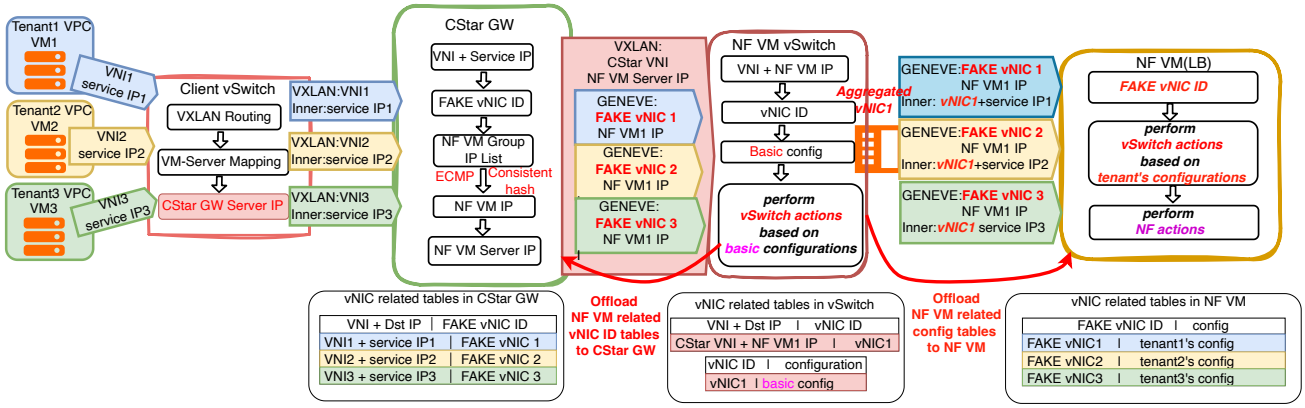

Figure 10: Packet journey from the tenant VM to the NF VM with CStar Gateway (GW) for vNIC aggregation at the NF VM.

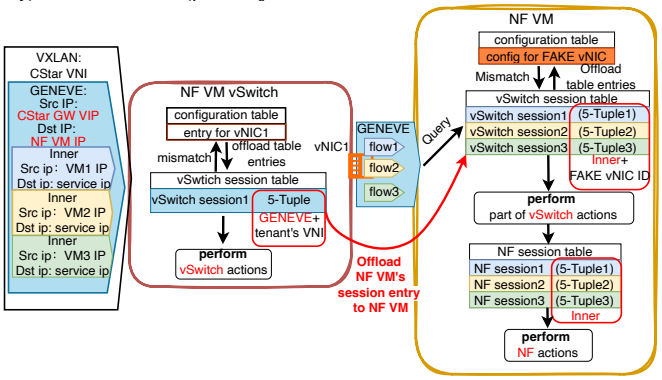

Figure 11: Aggregate session by GENEVE.

EVE header and an outer VXLAN header using a system-level CStar VNI. Consequently, the NF VM-side vSwitch tracks sessions using this CStar VNI(instead of each tenant's VNI) and the GENEVE five-tupleinstead of the original inner five-tuple). By tunneling all traffic destined for an NF VM through a single endpoint, concurrent flows collapse into one vSwitch session entry. This eliminates session table exhaustion; for instance (Fig. 11), three distinct flows consume only a single session slot.

Before aggregation, separate flows required individual session entries in the vSwitch for distinct per-tenant NF access control. After aggregation, the NF VM-side vSwitch applies uniform and basic actions(related to ACL and meter) to all flows in the same GENEVE tunnel. To maintain fine-grained multi-tenant, multi-demand control, specific per-tenant's demand actions must still be handled inside the NF VM, requiring that tenant-specific configurations be migrated from the vSwitch. The NF VM reuses the vSwitch's fast/slow path logic and records actions in its session entries, effectively offloading the entire session generation and storage burden from the vSwitch. This allows the vSwitch to aggregate flows while the NF VM handles their de-aggregation and performs NF access control.

As shown in Fig. 11, the session key differs between the vSwitch and NF VM. The vSwitch uses (tenant VNI + inner five-tuple). However, because the gateway's GENEVE encapsulation replaces the tenant VNI with the CStar system VNI, the NF VM cannot access it. To maintain tenant isolation, the NF VM substitutes the tenant VNI with a fake vNIC ID, which uniquely maps to a specific tenant' demand. Thus, the NF VM session key becomes (fake vNIC ID + inner five-tuple).

4.2 Handling TR Traffic and Elephant Flows with FPGA-Based NFs

Processing Elephant Flows with NF FPGA. For cost-efficiency and customization, we deploy programmable, line-rate FPGAs to handle elephant flows, referred to as NF FPGAs. However, due to their limited elasticity in matching dynamic demands, we adopt a hybrid architecture that combines NF FPGAs for elephant flows with elastic NF VMs for other workloads. By routing traffic based on flow characteristics, this design enables the system to get both high throughput and cloud-native elasticity.

To detect unpredictable elephant flows, NF VMs monitor real-time per-flow packet and byte counters. Upon detection (Fig. 12), the NF VM sends the flow identifier to the CStar Gateway via the control plane. The Gateway then redirects subsequent packets to an NF FPGA, leaving normal flows on NF VMs. This flow identifier is a composite key: (fake vNIC ID + hash value). The hash, pre-computed from the inner five-tuple during Gateway ECMP, is embedded directly into the GENEVE header as a stable ID. Prepending the fake vNIC ID prevents cross-tenant hash collisions.

To enable NF FPGAs to process elephant flows directly, the NF session entry containing the flow actions (*e.g.*, VIP-to-RS mapping) is migrated from the NF VM to the NF FPGA through the control plane after detection. Because elephant flows are identified mid-lifecycle rather than at the first packet, the NF VM already holds a valid session entry. Upon migration, the NF FPGA installs this entry and executes the same actions as the NF VM, ensuring functional equivalence and state continuity without disrupting flow processing.

For traffic analytics and billing, the NF FPGA collects per-flow statistics (*e.g.*, packet and byte counts) and reports

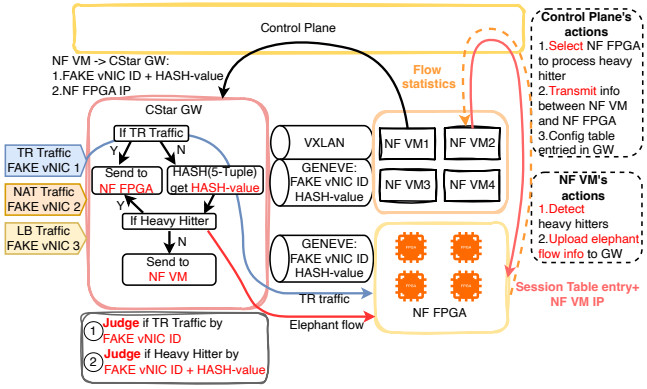

Figure 12: Offload TR and heavy hitter to FPGA.

them through the control plane to the NF VM that originally handled the flow. As shown in Fig. 12, the NF VM includes its own IP address when transferring the session entry to the NF FPGA to ensure correct delivery. The NF FPGA extracts the flow identifier from the session, pairs it with the NF VM's IP, and sends the annotated statistics through the control plane. These are then delivered to the target NF VM for statistic update.

Offloading I/O-intensive Traffic to FPGA. Although FPGA- and VM-based TR instances can achieve identical throughput, their computational resource allocation differs significantly. At the same line rate, VMs contain far more computational capacity (ops/sec) than required, whereas FPGA resources are concentrated on I/O processing. This I/O–optimized resource ratio makes FPGA a more efficient choice for TR workloads, which demand high throughput but minimal per-packet computation. In addition to improved resource alignment, NF FPGA incurs lower operational costs, directly enhancing the profit margin of TR services.

To steer TR traffic to NF FPGAs while keeping other NF traffic on NF VMs, we classify flows by service type. This is straightforward because tenants access NF services through assigned service IPs. The CStar Gateway deterministically identifies the traffic type using the tuple (Service IP, Tenant VNI). As shown in Fig. 12, this tuple maps to a fake vNIC ID that encodes tenant demand, including NF service type. The fake vNIC ID then enables the gateway to route TR-bound flows to NF FPGAs and all other flows to NF VMs.

4.3 Simplifying Elasticity via CStar Gateway

To eliminate the need for consistent hashing in the tenant-side vSwitch and its associated resource overhead and stability risks, we migrate ECMP load balancing for NF traffic to the CStar Gateway. As shown in Fig. 10, tenant-side vSwitches now simply forward all NF traffic to the CStar Gateway with no ECMP required. Within the gateway, ECMP is performed by first resolving the target NF VM group using the flow's fake vNIC ID, which uniquely maps to a tenant demand, and then applying consistent hashing to select a specific NF VM

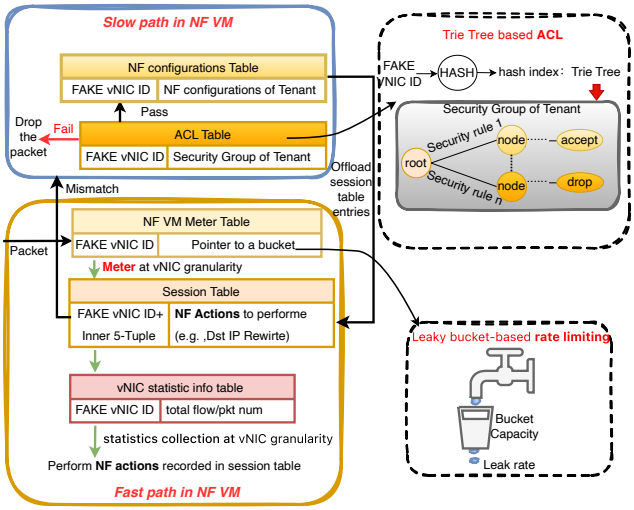

Figure 13: Implement ACL and Meter in NF VM.

and its host server. All of these processes are stateless.

Without the CStar Gateway, scaling an NF VM group required the controller to push ECMP updates to hundreds of geographically distributed tenant-side vSwitches, which is a slow and error-prone process. By centralizing ECMP logic in the gateway, the controller now updates only a few centralized tables in gateway. This eliminates distributed reconfigurations, thereby simplifying control-plane management, reducing load, and shortening scaling latency.

5 System Implementation

5.1 ACL and Meter in NF VM

With CStar Gateway, tenant configs (ACLs, metering) move from vSwitch to NF VM, eliminating vSwitch dependency. NF VM now enforces full per-tenant-demand NF access control, not just NF service logic. When performing NF-service logic, we introduce a software-based fast/slow path separation packet processing pipeline within the NF VM, achieving near-line-rate performance through careful design. To achieve near-line-rate processing, we need to insert ACL and metering functions into this existing pipeline as illustrated in Fig. 13.

ACL is inserted into the slow path, executing before any NF-specific logic. The steps are: 1) On the first packet, extract fake vNIC ID and retrieve associated security group rules; 2) Match inner 5-tuple against rules and drop immediately if denied; 3) Only if allowed, proceed to original NF slow-path logic and offload NF session entry to fast path. Placing ACL before NF slow-path logic ensures that only traffic permitted by security policies can access tenant NF configurations and trigger downstream processing. This effectively preserves the same vNIC-level NF traffic access control policies enforced by vSwitch and is now implemented entirely within the software data plane within NF VM. As shown in Fig. 13, to maximize ACL matching efficiency, we organize each tenant's security group rules into a dedicated trie tree. The

tree is indexed by fake vNIC ID, enabling O(k) prefix-based lookups per flow (k is the rule depth). To enforce vNIC-level rate limiting migrated from the vSwitch, NF VM firstly conducts metering in the fast path, before any further processing (ACL, NF logic). As shown in Fig.13, we adopt the leaky bucket algorithm, configured per fake vNIC ID with bucket capacity and drain rate. During initialization, NF VM pre-allocates a leaky bucket for each fake vNIC ID and builds a fast lookup table mapping fake vNIC ID → bucket pointer. After metering, NF VM can continue to process later processing logic (*e.g.*, look up session table, perform NF actions, get per-fake-vnic-id statistic).

Migrating per-vNIC configurations to the NF VM eliminates vSwitch pressure, but at the cost of additional NF VM memory. However, production telemetry shows that storing ACLs (as trie trees) and metering state (as leaky buckets) consumes only a few gigabytes per NF VM. Given that NF VMs are typically allocated tens of gigabytes of DPDK-managed hugepage memory, most of which remains unused. Memory exhaustion is not a practical concern, even under multi-tenant and high-rule-count workloads. NF VM forwarding operates in Run-To-Completion (RTC) mode, where each core independently executes the full data plane. Introducing ACL and metering adds measurable but bounded CPU overhead: 1) ACL (slow path): Applied per new flow, performs prefix matching via trie tree. This increases CPU utilization by 10–20% of baseline under high flow-setup rates (*e.g.*, 20% → 22–24%), but negligible under low rates. 2) Metering (fast path): Applied to all flows and adds a stable 10% baseline overhead due to per-packet leaky bucket updates. Since NF VMs were previously bottlenecked by vNIC/session limits instead of CPU, their cores typically ran at low utilization. Even with the added overhead, CPU usage remains well within safe operating margins, preserving headroom for scaling and burst handling.

5.2 Tofino based CStar Gateway

The CStar Gateway's data plane is inherently stateless and table-driven, involving only header parsing, table lookups, metadata tagging, and encapsulation. This is ideally suited for implementation on programmable switches like Tofino, which offer line-rate throughput and nanosecond-level per-packet processing. The core forwarding logic is decomposed into a sequence of match-action tables as shown in the Table 1: M/A table 1 is to get FAKE vNIC ID; M/A table 2 and 3 are to judge traffic type, if it is TR traffic or elephant flow, transmit the flow to an NF FPGA IP (value of table 2 and 3); M/A table 4 and 5 are used to ECMP all other flow to a selected NF VM in a group using consistent hash. To implement Maglev consistent hashing in P4, we use a flat register array to store all NF VM group mappings. Each group has a dedicated hash table of 1024 slots, but instead of separate tables, we merge them into one large array. The index is computed as shown

```
struct meta{
    bit<8>  group_id;  bit<32> flow_hash;   bit<10>  vm_id;       //consistent hash
    bit<32> nf_vm_ip;  bit<32> server_ip;
    bit<1>  encap_type;   //0: GENEVE, 1:VXLAN + GENEVE
}
register<bit<10>> consistent_hash_table {
    size = 262144;  // 256 * 1024, 256 NF-VM group, every group with 1024 slots
}
parser MyParser(packet, hdrs, meta){...}
control ingress(...){
    ...
        table fake_vnic_id_to_group_id{
        key={meta.fake_vnic_id:exact};
        actions(bit<8> VM_group_id){meta.group_id=VM_group_id;}
    }
    bit<18> flat_index = ((bit<18>)meta.group_id << 10) | (bit<18>)(meta.flow_hash%1024);
    bit<10> temp_vm_id;
    consistent_hash_table.read(temp_vm_id, flat_index);
    meta.vm_id=temp_vm_id;
    table nf_vm_id_to_nf_vm_ip_nf_server_ip{
        actions(...){meta.nf_vm_ip=temp_vm_ip; meta.server_ip=temp_server_ip;}
    }
    apply{
        ... // tables.apply()
        //encap geneve
        if (meta.encap_type==0){hdr.geneve.setValid(); hdr.vxlan.setInvalid();}
        //encap geneve and vxlan
        else {hdr.geneve.setValid(); hdr.vxlan.setValid();}
    }
}
```

Figure 14: P4 code to implement consistent hash in Tofino.

in Fig. 14, which maps each flow to a unique slot that stores NF VM information, enabling efficient ECMP lookup with minimal memory.

Table 1: Tables in Tofino: Entry Width and Entry Count.

M/A Table Name	Entry len(b)	Num
1.vni_service_to_fake_vnic_id	80	10W
2.TR_traffic_to_fpga_ip	56	3W
3.heavy_hitters_to_fpga_ip	88	128
4.fake_vnic_id_to_group_id	32	7W
5.nf_vm_id_to_ip_nf	74	1K

CStar Gateway is deployed at AZ granularity. We derive the entry width and total table size for each match-action table based on production-scale metrics, including total NF VM count, NF VM group count, tenant-service demand volume, TR tenant count, and peak concurrent elephant flows. Concrete values are summarized in the Table 1. Alibaba's Tofino switches feature four independent ingress/egress pipelines, each equipped with ~100 Mb of SRAM and ~10 Mb of DRAM. The CStar Gateway tables described above consume less than 20 MB total, occupying <20% of per-pipeline memory capacity. As a result, all four pipelines can operate in parallel under the current production scale. The design, with no pipeline folding required, fully utilizes the switch's maximum line-rate throughput, without memory-induced bottlenecks.

5.3 FPGA based NF

NF FPGA processes both TR traffic and elephant flows from all NFs. It must import fast-path session entries with their associated actions for elephant flows. Since different NF types define heterogeneous action sets (*e.g.*, NF1: action1, 2, 3; NF2: action2, 3, 4, 5), we design a unified session entry format. This format can express all types of action within a single, extensible table structure, enabling NF FPGA to execute any NF's

logic without per-service specialization. As shown in Fig. 15, the session entry format in NF FPGA uses a composite key: (fake vNIC ID + inner 5-tuple), with a value field encoding the union of all possible actions across NF types. Each action consists of two parts: 1) Enable bit (1/0): indicates whether to execute the action; 2) Operand field: holds parameters required for execution (*e.g.*, IPv4/6 for SNAT/DNAT, port for rewriting). Aggregating all actions, a single session entry fits within 2 KB, which is sufficient to cover all NF types and their associated data. We implement NF FPGA on FM8 FPGAs, which provide hundreds of MB of on-chip BRAM and 4×4 GB of external DDR memory. To balance capacity, cost, and access latency, BRAM stores runtime metadata and temporary variables, while the large session table is placed in DDR.

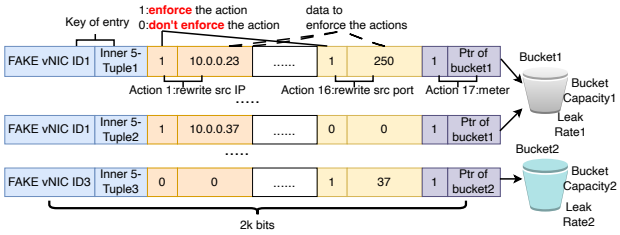

Figure 15: Fast-Path session entry format in NF FPGA.

To achieve higher per-instance bandwidth, we deploy four FPGA cards per server, forming a single logical NF FPGA instance. Each card delivers 200 Gbps of line-rate throughput, combining to an aggregate capacity of 800 Gbps for high-density NF processing. To process TR traffic, we adopt the hybrid execution: slow path (ACL + session install) runs on the host CPU and fast path (forwarding + metering) runs on the FPGA. Driven by unified 2 KB session entries pushed from the host, the FPGA fast path ensures functional parity with NF VMs at line rate. Unlike NF VMs, where metering is implemented as a standalone fast-path stage, metering is embedded in session entry as a pointer to a shared leaky bucket (per fake vNIC ID) to enable tenant-granular rate limiting, as shown in Fig. 15. To process elephant with zero-disruption handoff, we migrated the session entries for elephant flows from NF VM to FPGA via the controller. Within the FPGA's fast path, beyond forwarding based on session entries, the FPGA also needs to collect per-fake-vNIC-ID flow stats (pkts/bytes) and report back to NF VM. These are used by cloud providers for billing and logging.

6 Evaluation

6.1 Experimental setting

CStar Gateway and NF FPGA are deployed at AZ granularity. An NF FPGA instance (800 Gbps) is deployed to handle all TR and elephant flows within an AZ. Two identical Tofino-based CStar Gateway instances are deployed per AZ, with one serving as redundancy. The gateways share a single VIP, and the underlay network performs ECMP load balancing

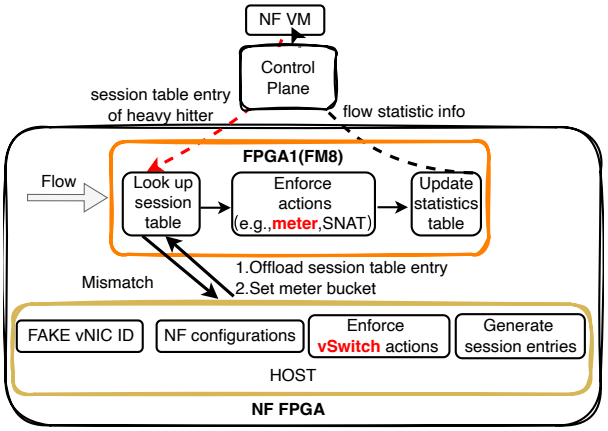

Figure 16: Processing pipeline of NF FPGA.

across both. Notably, this architecture has been running stably in Alibaba Cloud for nearly two years.

6.2 Experimental results

The number of tenant NF demands served per NF VM can be determined by vNIC count (without gateway) or fake vNIC ID count (with gateway). As shown in Fig. 17, the number of tenant NF demands served per NF VM increased dramatically across all three AZs after deploying CStar Gateway. The improvement varies by AZ (11×, 11×, 5×). The variation reflects differences in NF VM type composition across AZs. We also observe that, by removing the vNIC quota bottleneck, a single VPC NAT NF VM can serve up to 7K tenant demands.

As shown in Fig. 18, we measure the concurrent flow count on a single L4-LB NF VM over 24 hours. Without CStar Gateway (blue curve), flow count plateaus at ~15M during peak hours (10:00–22:00), which is strictly capped by vSwitch session table limits. With CStar Gateway (orange curve), flow scale surges to 40M+. Instead of being constrained by the session table quota on the vSwitch, this number is influenced by the NF VM's own bandwidth, memory, and CPU capacity. This scaling gain is achieved without VM resizing or hardware upgrade, demonstrating CStar Gateway's ability to unlock latent capacity in existing NF VMs.

We measure the scaling latency from trigger (at 9 Gbps) to completion for expanding an NF VM group from 4 to 8 instances. As shown in Fig. 19, without CStar Gateway, the process takes ~11 seconds, due to distributed vSwitch state updates. With CStar Gateway, scaling completes in less than 2 seconds, achieving a 5× improvement. This scaling process involves only controller-to-gateway communication, eliminating vSwitch reconfiguration overhead.

We measure end-to-end latency from a tenant VM to a VPC NAT NF VM across packet sizes from 256B to 1500B, with CStar Gateway on the data path. As shown in Fig. 20, overall latency (blue curve) exceeds 290μs. The CStar Gateway's processing latency (orange curve) remains under 1.2μs, which is less than 0.4% of the total. These results show that

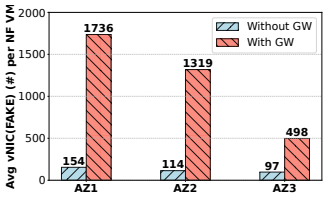

Figure 17: Avg num of FAKE vNIC/vNIC per NF VM with/without gateway (GW).

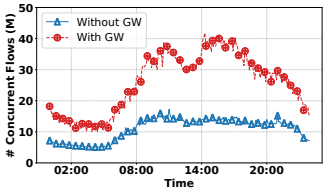

Figure 18: Concurrent flow per LB NF VM held with/without gateway.

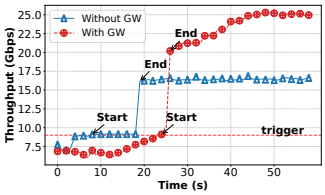

Figure 19: Elastic process LB with/without gateway.

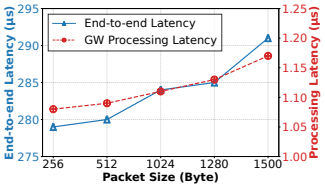

Figure 20: Latency of transmission and gateway processing under different packet sizes.

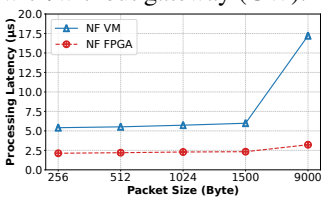

Figure 21: Processing latency of NF VM and NF FPGA.

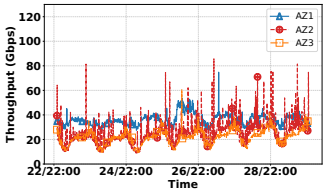

Figure 22: Throughput of NF FPGA in 3 AZs.

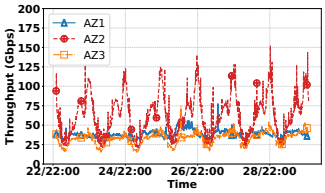

Figure 23: Throughput of CStar Gateway in 3 AZs.

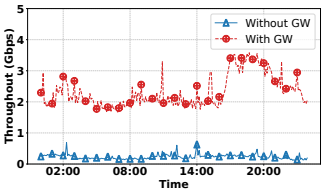

Figure 24: Throughput of one VPC NAT NF VM group with/without gateway.

CStar Gateway introduces negligible overhead, which can be transparent to tenant latency SLOs.

Fig. 21 compares fast-path per-packet latency (excluding first-packet setup) between VM-based and FPGA-based TR instances across packet sizes. FPGA processing (orange) is consistently $\geq$2.5 μs faster than VM (blue), demonstrating its higher data-plane efficiency.

Fig. 22 and Fig. 23 show one-week (8/22 22:00 – 8/29 22:00) traffic telemetry across three AZs. Despite elephant flow bursts pushing single-AZ FPGA traffic beyond 200 Gbps, our 800 Gbps-capable NF FPGA instances operate at <25% utilization, leaving ample space for future growth. Similarly, each CStar Gateway (multi-Tbps capacity) handles <250 Gbps peak (<10% of its limit), enabling further NF VM deployment without gateway scaling. TR traffic carried entirely by FPGA accounts for nearly 50% of all NF traffic volume.

Fig. 24 and Fig. 25 compare the daily throughput of two NF VM groups (8 NF VM VPC NAT group and 4 NF VM L4-LB group) without and with gateway. Without gateway (blue), both groups are severely throttled by vNIC/session limits. Specifically, the VPC NAT group is <0.7 Gbps. Despite 2× more VMs, its total throughput is lower than LB, with lightweight per-tenant flows. L4-LB group is $\leq$3.0 Gbps, constrained by session table exhaustion under high flow concurrency. With CStar Gateway (orange), the vNIC/session limits are removed. VPC NAT throughput increases fivefold to 3.5 Gbps without vNIC limits. L4-LB increases nearly sixfold to 17.5 Gbps without session limits. With the same VM count and specification, each VM now carries 5× more traffic.

As shown in Fig. 26 (I/O Utilization), all three AZs show a $\geq$5× increase in average NF VM I/O utilization after deploying the CStar Gateway. Without the gateway, NF VMs were underutilized due to vNIC and session limits. In Fig. 27 (CPU Utilization), CPU usage also rises by $\geq$5×, driven by increased per-VM traffic volume and the addition of in-VM ACL and metering logic in the data plane. Since vNIC-level configurations and session tables are offloaded to NF VMs, we also evaluate memory consumption. As shown in Fig. 28 (Memory Utilization), memory usage increases by less than 4.3% on average, indicating that this trade-off is efficient.

Table 2: The number of NF VMs allocated in the AZs before and after the deployment of CStar Gateway.

	AZ1	AZ2	AZ3
Without CStar Gateway	92	124	127
With CStar Gateway	18	7	11

After the deployment of CStar Gateway, NF VMs are able to handle more traffic. As a result, fewer NF VMs are required for the same amount of workload, leading to a reduction in NF VM procurement cost. Table 2 shows the number of NF VMs required in AZs before and after deploying CStar Gateway. Even after considering the cost of the CStar Gateway itself, our expenses in the three AZs have reduced by 71.91%, 88.57%, and 85.69%, respectively.

7 Experience and Lessons

Single-Tenant I/O Starves Shared vNIC. After deployment, tenants occasionally reported NF service unreachability. Investigation revealed that during these incidents, over 99% of the aggregated vNIC bandwidth was consumed by traffic from a single fake vNIC ID, blocking flows from other tenants. The root cause was the location of rate control. While rate limiting was moved into NF VMs to enforce per-tenant SLAs and protect CPU/memory resources, NF VMs cannot control the arrival rate at the shared vNIC. Ingress rate control remained with the vSwitch, which lacked visibility into fake vNIC IDs and thus could not conduct tenant-granular limits. As a result,

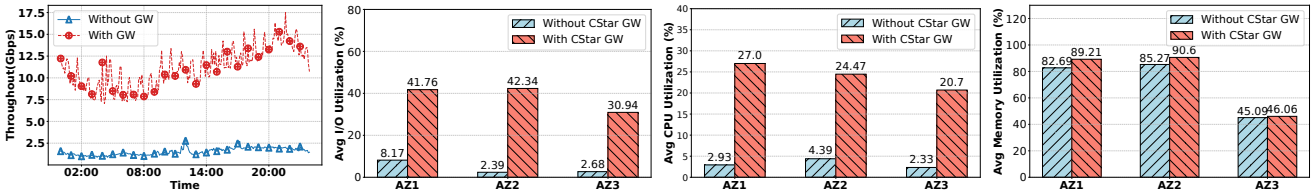

Figure 25: Throughput of one LB NF VM group with/without gateway (GW).

Figure 26: NF VM I/O utilization improvement with/without gateway.

Figure 27: NF VM CPU utilization improvement with/without gateway.

Figure 28: NF VM memory utilization improvement with/without gateway.

a single tenant could monopolize the shared vNIC's ingress bandwidth.

To mitigate this, we introduced coarse-grained rate limiting in the CStar Gateway. Traffic is first aggregated per NF VM, and if the total ingress rate exceeds a configured threshold, the gateway applies per–fake-vNIC-ID rate limiting for all flows mapped to that NF VM. This prevents any single tenant from saturating the shared vNIC and restores fairness at the entry point. However, the approach is not scalable. Each fake vNIC ID requires a dedicated leaky bucket, consuming scarce Tofino register memory. While feasible at the current scale (with spare capacity), accommodating larger tenant populations introduces scalability risks, motivating our future research on memory-efficient alternatives.

RSS Skew on vSwitch in GENEVE. After using CStar GW, we observed a higher frequency of single-core hotspot events on vSwitches. Further analysis revealed that affected vSwitches almost always hosted one or more NF VMs. The root cause lies in how RSS is performed after VXLAN decapsulation. Without the gateway, the vSwitch applies RSS using the inner 5-tuple, which distributes flows evenly across cores. With the gateway, however, RSS is performed on the uniform GENEVE outer 5-tuple. As a result, flows are often mapped to the same vSwitch core, leading to single-core overload. To restore load balance, we randomized the GENEVE UDP source port on a per-flow basis. Since the UDP destination port must remain fixed for GENEVE demultiplexing, modifying the source port provides the necessary tuple diversity to spread flows evenly across vSwitch cores. This workaround requires no changes to vSwitch RSS logic or hardware, making it a practical and production-safe solution.

Why GENEVE? We chose GENEVE over alternatives (*e.g.*, VXLAN-GPE, MPLS) for two key reasons. First, GENEVE natively supports extensible metadata through its TLV-based options, allowing us to embed flow-related context, such as fake vNIC IDs, which we assign a 24-bit size and position within the option section of the GENEVE layer encapsulation. This specific placement is designed to facilitate future modifications to the fake vNIC ID's bit width as requirements evolve. We also use these options for other contexts, like per-flow hash values for elephant-flow handling, without redesigning the header. This extensibility also makes it easy to accommodate future needs by carrying additional control information in-band. Second, GENEVE was already the default encapsulation for NF-to-NF communication (e.g., chaining VPC NAT with TR). As a result, NF VMs already support GENEVE. By adopting GENEVE in CStar Gateway, we avoided redundant development effort and established a unified metadata plane across the CStar Gateway, NF VM, and inter-NF paths, ensuring architectural consistency and long-term flexibility.

Metering as Separate Stage vs. in Session. Although metering is implemented in the fast path in both NF FPGA and NF VM, their realization is different. In NF FPGA, the metering is embedded as an action within each session entry. In NF VM, it is implemented as a standalone pipeline stage. This design choice was driven primarily by development effort. Given the severe resource mismatches faced by the CStar platform, we prioritized rapid functionality rollout. NF VMs deployed in production exhibit heterogeneous session entry formats; embedding the meter into entries would require per–NF VM adaptation. Implementing it as an independent pipeline stage allowed code reuse across multiple DPDK-based NF VMs, significantly reducing engineering overhead. By contrast, Alibaba's FPGA-based fast paths already included this design pattern, where meters were attached to session entries as actions. Reusing this pattern enabled rapid integration while minimizing development cost.

Why We Use (fake vNIC ID, hash value) not 5-Tuple to Identify Elephant Flows. We identify each elephant flow using a combination of the flow's hash value and its fake vNIC ID. The hash value is computed once at the CStar Gateway and then conveyed to NF VMs through GENEVE. We avoid recomputing the hash at NF VMs, since their existing hash functions differ from that of the gateway. Enforcing consistency would require intrusive modifications across NF VMs. We also avoid using the full 5-tuple as the identifier. First, NF VMs need to report the identifier back to the gateway via the control plane, and transmitting a compact hash value has lower overhead than carrying a full 5-tuple. Second, matching 5-tuples would require the gateway to parse inner packet headers and maintain larger match tables, consuming more hardware resources. For these reasons, we use both the hash value and fake-vNIC-ID as the practical and resource-efficient identifier for elephant flows.

Rethinking Resource Utilization: A New Paradigm for Cost-Effective Cloud NFV. This solution offers a novel approach for all vendors utilizing VMs for NF processing. It ap-

plies the "Buckets Effect" metaphor, where the entire service-providing VM is viewed as a bucket, its integrated available resources constitute the planks, and the maximum NF traffic capacity represents the bucket's total volume. To increase the NF traffic capacity of a single VM, beyond merely upgrading VM specifications to increase the overall "bucket" size, one can also optimize the architectural design to alter resource utilization patterns. Without sacrificing functionality, this approach eliminates critical bottlenecks (the "shortest planks") that previously constrained capacity. Consequently, the maximum capacity is determined by other, less restrictive resources (the "longer planks"), effectively expanding the per-VM throughput. This design methodology enables cloud providers to handle significantly higher traffic volumes using the same VM specifications. This directly reduces traffic processing costs while increasing revenue, aligning perfectly with the economic and efficiency demands of cloud operators.

CStar Gateway Capacity Planning and Elastic Scaling Strategy. Regarding the elasticity of the CStar Gateway itself, we provide the order of magnitude for its table entry storage in Table 1. Under the current traffic scale, a single CStar Gateway to handle all NF traffic in an AZ occupies less than 20% of the Tofino chip's storage and less than 10% of its throughput even at peak. A single CStar Gateway can handle a significantly higher NF traffic load within its existing footprint. Therefore, we do not currently consider the issue of elastic expansion for the CStar Gateway within an AZ. In the event of future storage or throughput limitations, our strategy, consistent with other hardware gateway practices, is not to expand the hardware capacity of the existing CStar Gateway in that AZ, given that physical machine expansion is complex and time-consuming. Instead, we would migrate any NF traffic that exceeds the CStar Gateway's capacity to a new AZ with abundant available CStar Gateway resources.

High Availability and Failure Recovery Mechanisms We implement distinct high availability (HA) mechanisms for both the CStar Gateway and the NF VMs. For the CStar Gateway, we employ an active-active disaster recovery design. Within each Availability Zone (AZ), we deploy two CStar Gateways, and the control plane keeps their forwarding tables perfectly synchronized. These two gateways expose a single, unified Virtual IP (VIP), and the underlying physical network uses Equal-Cost Multi-Path (ECMP) routing to distribute all tenant traffic bound for NF VMs across both gateways. This active-active mechanism ensures high system stability; upon a single gateway failure, all traffic seamlessly shifts to the remaining gateway with negligible packet loss. Regarding NF VM failures, our approach involves redirecting all traffic from the failed VM to a healthy VM with sufficient remaining resources. Unlike previous failure handling methods that only required migrating traffic sessions to the new VM, our current architecture necessitates simultaneously migrating the tenant-associated security group rules and rate-limiting configurations from the failed VM to the new one. Furthermore, we must update all entries in the CStar Gateway's ECMP tables, replacing the failed VM's address with the new VM's IP address to ensure proper traffic steering.

8 Related work

Compared to hardware middleboxes [12], NFV offers service flexibility, cost efficiency, and resource optimization for public cloud vendors. However, achieving line-rate packet processing and horizontal scalability [10, 13, 21, 22, 24, 27] pose significant challenges for NFV implementations. Azure [4] initially enhanced single-node processing by deploying SmartNICs on servers [8] within their VPC infrastructure. However, this approach reduced NFV flexibility and led to resource underutilization. To address this, they adopted a shared resource pool design, which reduced the per-server processing capability and redirected overflow traffic to the pools. Google's Andromeda [5] implemented complex NFV functionality entirely in software VMs. However, its architecture ensures that each tenant is allocated a dedicated VM to execute network functions. They do not support the scenario where multiple tenants share the same NF VM. While prior NFV systems scale by adding hardware or VMs, our work tackles this resource imbalance using existing NF VMs. We offload vSwitch's overhead (vNICs, sessions) into underutilized NF VMs, and selectively accelerate heavy flows (e.g., elephant flows, TR) via FPGA to preserve elasticity and performance. Prior NFV systems generally scale by adding hardware or VMs. In contrast, our work addresses resource imbalance within existing NF VMs. We offload vSwitch's overhead (vNICs, sessions) into underutilized NF VMs, while selectively accelerating heavy flows using the FPGA. This approach preserves both elasticity and performance without requiring additional hardware or VM.

9 Conclusion

In this work, we introduce a design combined with the Tofino-based gateway and FPGA to solve resource mismatch problems when building scalable NFV products using standard VMs. This solution significantly reduces the cost of deploying NFV systems on public clouds based on standard VMs, without requiring intrusive changes to the underlying cloud virtualization architecture. It offers a viable path for other cloud service providers seeking to deliver cost-effective and cloud-based NFV services.

Acknowledgments

The authors would like to thank the shepherd Chenxi Wang and the anonymous reviewers for their constructive comments. This work was in part supported by Key R&D Program of Zhejiang Province (No. 2023R5202) and Alibaba Research Intern Project.

References

[1] Alibaba cloud transit router. https://www.alibabacloud.com/en/product/transit_router, 2025.

[2] Features and metric data of ecs instance families. https://www.alibabacloud.com/help/en/ecs/user-guide/overview-of-instance-families, 2025.

[3] Deepak Bansal, Gerald DeGrace, Rishabh Tewari, Michal Zygmunt, James Grantham, Silvano Gai, Mario Baldi, Krishna Doddapaneni, Arun Selvarajan, Arunkumar Arumugam, et al. Disaggregating stateful network functions. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 1469–1487, 2023.

[4] Deepak Bansal, Gerald DeGrace, Rishabh Tewari, Michal Zygmunt, James Grantham, Silvano Gai, Mario Baldi, Krishna Doddapaneni, Arun Selvarajan, Arunkumar Arumugam, et al. Disaggregating stateful network functions. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 1469–1487, 2023.

[5] Michael Dalton, David Schultz, Jacob Adriaens, Ahsan Arefin, Anshuman Gupta, Brian Fahs, Dima Rubinstein, Enrique Cauich Zermeno, Erik Rubow, James Alexander Docauer, et al. Andromeda: Performance, isolation, and velocity at scale in cloud network virtualization. In *15th USENIX symposium on networked systems design and implementation (NSDI 18)*, pages 373–387, 2018.

[6] Daniel E Eisenbud, Cheng Yi, Carlo Contavalli, Cody Smith, Roman Kononov, Eric Mann-Hielscher, Ardas Cilingiroglu, Bin Cheyney, Wentao Shang, and Jinnah Dylan Hosein. Maglev: A fast and reliable software network load balancer. In *Nsdi*, volume 16, pages 523–535, 2016.

[7] Daniel Firestone. VFP: A Virtual Switch Platform for Host SDN in the Public Cloud. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 315–328, Boston, MA, March 2017. USENIX Association.

[8] Daniel Firestone, Andrew Putnam, Sambhrama Mundkur, Derek Chiou, Alireza Dabagh, Mike Andrewartha, Hari Angepat, Vivek Bhanu, Adrian Caulfield, Eric Chung, Harish Kumar Chandrappa, Somesh Chaturmohta, Matt Humphrey, Jack Lavier, Norman Lam, Fengfen Liu, Kalin Ovtcharov, Jitu Padhye, Gautham Popuri, Shachar Raindel, Tejas Sapre, Mark Shaw, Gabriel Silva, Madhan Sivakumar, Nisheeth Srivastava, Anshuman Verma, Qasim Zuhair, Deepak Bansal, Doug Burger, Kushagra Vaid, David A. Maltz, and Albert Greenberg. Azure accelerated networking: SmartNICs in the public cloud. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pages 51–66, Renton, WA, April 2018. USENIX Association.

[9] Rohan Gandhi, Hongqiang Harry Liu, Y Charlie Hu, Guohan Lu, Jitendra Padhye, Lihua Yuan, and Ming Zhang. Duet: Cloud scale load balancing with hardware and software. *ACM SIGCOMM Computer Communication Review*, 44(4):27–38, 2014.

[10] Aaron Gember-Jacobson, Raajay Viswanathan, Chaithan Prakash, Robert Grandl, Junaid Khalid, Sourav Das, and Aditya Akella. Opennf: Enabling innovation in network function control. *ACM SIGCOMM Computer Communication Review*, 44(4):163–174, 2014.

[11] J Gross, I Ganga, and T Sridhar. Rfc 8926: Geneve: Generic network virtualization encapsulation, 2020.

[12] Masoud Hemmatpour, Changgang Zheng, and Noa Zilberman. E-Commerce Bot Traffic: In-Network Impact, Detection, and Mitigation. In *2024 27th Conference on Innovation in Clouds, Internet and Networks (ICIN)*, pages 179–185, 2024.

[13] Murad Kablan, Azzam Alsudais, Eric Keller, and Franck Le. Stateless network functions: Breaking the tight coupling of state and processing. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 97–112, Boston, MA, March 2017. USENIX Association.

[14] David Karger, Eric Lehman, Tom Leighton, Rina Panigrahy, Matthew Levine, and Daniel Lewin. Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the world wide web. In *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*, pages 654–663, 1997.

[15] Praveen Kumar, Nandita Dukkipati, Nathan Lewis, Yi Cui, Yaogong Wang, Chonggang Li, Valas Valancius, Jake Adriaens, Steve Gribble, Nate Foster, and Amin Vahdat. Picnic: predictable virtualized nic. In *Proceedings of the ACM Special Interest Group on Data Communication*, pages 351–366. 2019.

[16] Jianyuan Lu, Tian Pan, Shan He, Mao Miao, Guangzhe Zhou, Yining Qi, Biao Lyu, and Shunmin Zhu. A two-stage heavy hitter detection system based on cpu spikes at cloud-scale gateways. In *2021 IEEE 41st International Conference on Distributed Computing Systems (ICDCS)*, pages 348–358. IEEE, 2021.

[17] Colm MacCarthaigh. Multi-tier stateful network flow management architecture, June 12 2018. US Patent 9,998,955.

[18] Colm MacCárthaigh. Workload isolation using shuffle-sharding. *Tech. Rep.*, 2019.

[19] Rui Miao, Hongyi Zeng, Changhoon Kim, Jeongkeun Lee, and Minlan Yu. Silkroad: Making stateful layer-4 load balancing fast and cheap using switching asics. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, pages 15–28, 2017.

[20] Tian Pan, Nianbing Yu, Chenhao Jia, Jianwen Pi, Liang Xu, Yisong Qiao, Zhiguo Li, Kun Liu, Jie Lu, Jianyuan Lu, Enge Song, Jiao Zhang, Tao Huang, and Shunmin Zhu. Sailfish: Accelerating cloud-scale multi-tenant multi-service gateways with programmable switches. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, pages 194–206, 2021.

[21] Parveen Patel, Deepak Bansal, Lihua Yuan, Ashwin Murthy, Albert Greenberg, David A. Maltz, Randy Kern, Hemant Kumar, Marios Zikos, Hongyu Wu, Changhoon Kim, and Naveen Karri. Ananta: Cloud scale load balancing. *ACM SIGCOMM Computer Communication Review*, 43(4):207–218, 2013.

[22] Shriram Rajagopalan, Dan Williams, Hani Jamjoom, and Andrew Warfield. Split/Merge: System support for elastic execution in virtual middleboxes. In *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*, pages 227–240, Lombard, IL, April 2013. USENIX Association.

[23] Kyle Schomp, Onkar Bhardwaj, Eymen Kurdoglu, Mashooq Muhaimen, and Ramesh K Sitaraman. Akamai dns: Providing authoritative answers to the world's queries. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, pages 465–478, 2020.

[24] Justine Sherry, Peter Xiang Gao, Soumya Basu, Aurojit Panda, Arvind Krishnamurthy, Christian Maciocco, Maziar Manesh, João Martins, Sylvia Ratnasamy, Luigi Rizzo, and Scott Shenker. Rollback-recovery for middleboxes. volume 45, page 227–240, New York, NY, USA, August 2015. Association for Computing Machinery.

[25] Justine Sherry, Shaddi Hasan, Colin Scott, Arvind Krishnamurthy, Sylvia Ratnasamy, and Vyas Sekar. Making middleboxes someone else's problem: network processing as a cloud service. *SIGCOMM Comput. Commun. Rev.*, 42(4):13–24, August 2012.

[26] Enge Song, Nianbing Yu, Tian Pan, Qian Fu, Liang Xu, Xionglie Wei, Yisong Qiao, Jianyuan Lu, Yijian Dong, Mingxu Xie, Jun He, Jinkui Mao, Zheng-Jun Luo, Chenhao Jia, Jiao Zhang, Tao Huang, Biao Lyu, and Shunmin Zhu. MIMIC: SmartNIC-aided Flow Backpressure for CPU Overloading Protection in Multi-Tenant Clouds. pages 1–11, 2022.

[27] Shinae Woo, Justine Sherry, Sangjin Han, Sue Moon, Sylvia Ratnasamy, and Scott Shenker. Elastic scaling of stateful network functions. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pages 299–312, 2018.

[28] Tingting Xu, Bengbeng Xue, Yang Song, Xiaomin Wu, Xiaoxin Peng, Yilong Lyu, Xiaoliang Wang, Chen Tian, Baoliu Ye, Camtu Nguyen, Biao Lyu, Rong Wen, Zhigang Zong, and Shunmin Zhu. CyberStar: Simple, elastic and Cost-Effective network functions management in cloud network at scale. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 227–246, Santa Clara, CA, July 2024. USENIX Association.

[29] Mingyuan Zang, Changgang Zheng, Lars Dittmann, and Noa Zilberman. Toward Continuous Threat Defense: in-Network Traffic Analysis for IoT Gateways. *IEEE Internet of Things Journal*, 11(6):9244–9257, 2024.

[30] Chaoliang Zeng, Layong Luo, Teng Zhang, Zilong Wang, Luyang Li, Wenchen Han, Nan Chen, Lebing Wan, Lichao Liu, Zhipeng Ding, Xiongfei Geng, Tao Feng, Feng Ning, Kai Chen, and Chuanxiong Guo. Tiara: A Scalable and Efficient Hardware Acceleration Architecture for Stateful Layer-4 Load Balancing. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 1345–1358, Renton, WA, April 2022. USENIX Association.

[31] Changgang Zheng, Haoyue Tang, Mingyuan Zang, Xinpeng Hong, Aosong Feng, Leandros Tassiulas, and Noa Zilberman. DINC: Toward Distributed In-Network Computing. *Proc. ACM Netw.*, 1(CoNEXT3), November 2023.

Appendices

A Additional Experiences

A.1 CStar Gateway Is Stateless

A core design principle of CStar Gateway is keeping the gateway stateless: unlike NF VMs, which maintain per-flow session state, the gateway performs only table-based, metadata-driven forwarding. It stores no flow state, maintains no connection tables, and has no concept of fast/slow path. By being stateless, we can deploy multiple identical gateway instances per AZ, fronted by a single VIP and load-balanced via ECMP. This architecture delivers three critical production benefits: (1) horizontal scaling under traffic surge, (2) seamless failover during hardware/software faults, and (3) safe, incremental upgrades via gray deployment, all without service disruption.

A.2 Method to handle elephant flow before using FPGA

Before using CStar Gateway and NF FPGA, our elephant flow mitigation relied entirely on NF VMs and vSwitches: cores were monitored for saturation; top flows by packet rate were identified; and tenant-side vSwitches were instructed to rate-limit those flows(backpressure) to relieve CPU load. While functional in lab environments, this approach proved unacceptable in production: the sudden rate limiting at vSwitch caused bursty packet drops, violating tenant SLAs and destabilizing latency-sensitive applications. This forced a fundamental redesign: we offloaded elephant flow handling to NF-FPGA to eliminate packet loss during throttling, ensuring SLA compliance even under Gbps-scale bursts.

A.3 Why not implement consistent hash in vSwitch

Consistent hashing is typically deployed in overlay (not underlay) due to hardware constraints, but we do not implement it in vSwitch, even for NF-VM scaling. The reasons are:(1) maintaining per-group hash tables offers no benefits for general tenant VMs and no other workload except NF traffic requires it. Making dedicated consistent hashing support would cause a waste of memory and control-plane capacity. (2) vSwitch is a foundational multi-tenant component. Adding consistent hashing to vSwitches increases the probability of failure. Since vSwitch failures inherently impact a wide range of services, any failure can result in severe, widespread problems.

A.4 Optimizing Tofino Memory Usage

Implementing gateways on Tofino faces one hard constraint: on-chip memory (SRAM, TCAM, Register) is finite and precious [31]. While our deployment hasn't hit limits, we designed CStar Gateway with memory efficiency as a first-class goal. A representative optimization is that our consistent hash table (backed by register arrays) stores only an NF VM identifier (10 bits per slot) and not the full 64 bit (NF VM IP, server IP) tuple. Because different slots frequently map to the same backend VM, this indirection reduces memory consumption. A small, static secondary table then maps VM ID to network addresses, which adds minimal latency while enabling far greater scale.

A.5 How NF FPGA Achieves Elasticity for TR Traffic?

Although NF FPGA lacks the elastic agility of VMs, we deploy it for TR workloads because VM-based scaling proves economically unsustainable due to I/O intensity. To mitigate FPGA's elasticity limitation, we implement a two-pronged strategy: (1) Each NF FPGA instance supports up to 800 Gbps which exceeds current AZ-wide TR demand and absorbs short-term bursts without scaling; (2) We employ predictive traffic analytics and a tunable scaling threshold to trigger instance provisioning before capacity exhaustion which ensures zero packet loss or latency spike during scale events. This design mitigates FPGA's inability to handle elastic TR traffic by introducing limited elasticity, narrowing the gap between performance and elasticity.

B Mitigate the problem by enhancing the performance of the VM.

During the stage of system evolution, along with performance improvements in the VMs and the vSwitches that provide vNICs for them, the resources available per VM, such as the number of vNICs and sessions, have increased. We previously used this method to alleviate vNIC and session shortage issues in our systems that use VMs to provide NF services. The performance specifications of the several generations of VMs we used are shown in the Table B1; however, this approach did not fundamentally solve the problem.

Table B1: Performance comparison of different types of VMs.

	I/O	CPU	session	vNIC
Type 1	x	x	x	x
Type 2	2x	1.78x	1.94x	2x
Type 3	4x	2.78x	3.33x	4.17x
Type 4	8x	3.33x	6.67x	6.67x